

## Hough Line Transform

Prev Tutorial: [Canny Edge Detector](#)

Next Tutorial: [Hough Circle Transform](#)

### Goal

In this tutorial you will learn how to:

- Use the OpenCV functions [HoughLines\(\)](#) and [HoughLinesP\(\)](#) to detect lines in an image.

### Theory

#### Note

The explanation below belongs to the book **Learning OpenCV** by Bradski and Kaehler.

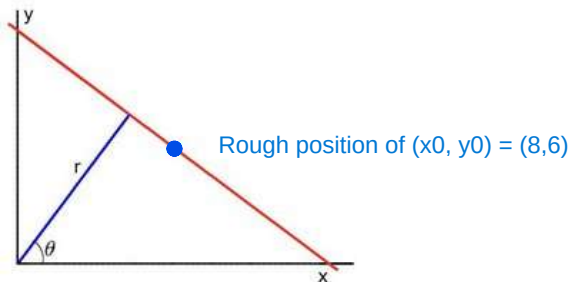
### Hough Line Transform

- The Hough Line Transform is a transform used to detect straight lines.
- To apply the Transform, first an edge detection pre-processing is desirable.

#### How does it work?

- As you know, a line in the image space can be expressed with two variables. For example:

- In the **Cartesian coordinate system**: Parameters:  $(m, b)$ .
- In the **Polar coordinate system**: Parameters:  $(r, \theta)$



For Hough Transforms, we will express lines in the *Polar system*. Hence, a line equation can be written as:

$$y = \left( -\frac{\cos \theta}{\sin \theta} \right) x + \left( \frac{r}{\sin \theta} \right)$$

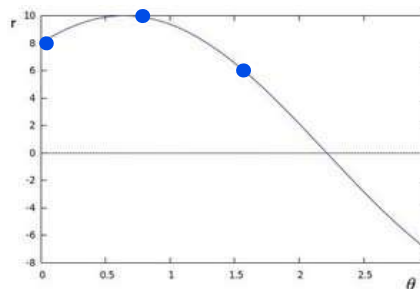
Arranging the terms:  $r = x \cos \theta + y \sin \theta$

- In general for each point  $(x_0, y_0)$ , we can define the family of lines that goes through that point as:

$$r_\theta = x_0 \cdot \cos \theta + y_0 \cdot \sin \theta$$

Meaning that each pair  $(r_\theta, \theta)$  represents each line that passes by  $(x_0, y_0)$ .

- If for a given  $(x_0, y_0)$  we plot the family of lines that goes through it, we get a sinusoid. For instance, for  $x_0 = 8$  and  $y_0 = 6$  we get the following plot (in a plane  $\theta - r$ ):



Search has identified  $(x_0, y_0) = (8, 6)$  as a point on line to be detected.

Now we vary Rho and Theta to create auxiliary lines for the Hough plane (accumulator array):

$$\text{Theta} = 0 \rightarrow \text{Rho} = 8 \cos(0) + 6 \sin(0) = 8$$

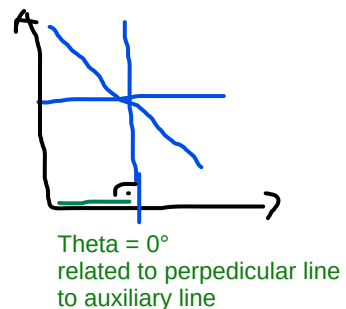
$$\text{Theta} = \text{Phi}/4 \rightarrow \text{Rho} = 8 \cos(\text{Phi}/4) + 6 \sin(\text{Phi}/4) = 9.9$$

$$\text{Theta} = \text{Phi}/2 \rightarrow \text{Rho} = 8 \cos(\text{Phi}/2) + 6 \sin(\text{Phi}/2) = 6$$

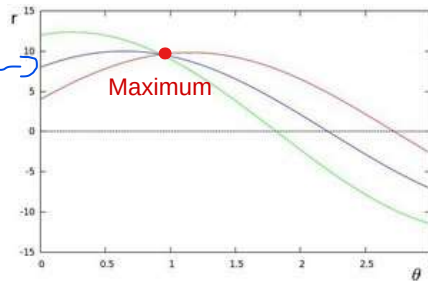
etc.

We consider only points such that  $r > 0$  and  $0 < \theta < 2\pi$ .

- We can do the same operation above for all the points in an image. If the curves of two different points intersect in the plane  $\theta - r$ , that means that both points belong to a same line. For instance, following with the example above and drawing the plot for two more points:  $x_1 = 4$ ,  $y_1 = 9$  and  $x_2 = 12$ ,  $y_2 = 3$ , we get:



Auxiliary lines for  
 $(x_0, y_0) = (8, 6)$



Maximum shows rho and theta  
of line to be detected

The three plots intersect in one single point  $(0.925, 9.6)$ , these coordinates are the parameters  $(\theta, r)$  or the line in which  $(x_0, y_0)$ ,  $(x_1, y_1)$  and  $(x_2, y_2)$  lay.

4. What does all the stuff above mean? It means that in general, a line can be *detected* by finding the number of intersections between curves. The more curves intersecting means that the line represented by that intersection have more points. In general, we can define a *threshold* of the minimum number of intersections needed to *detect* a line.
5. This is what the Hough Line Transform does. It keeps track of the intersection between curves of every point in the image. If the number of intersections is above some *threshold*, then it declares it as a line with the parameters  $(\theta, r_\theta)$  of the intersection point.

## Standard and Probabilistic Hough Line Transform

OpenCV implements two kind of Hough Line Transforms:

### a. The Standard Hough Transform

- It consists in pretty much what we just explained in the previous section. It gives you as result a vector of couples  $(\theta, r_\theta)$
- In OpenCV it is implemented with the function `HoughLines()`

### b. The Probabilistic Hough Line Transform

- A more efficient implementation of the Hough Line Transform. It gives as output the extremes of the detected lines  $(x_0, y_0, x_1, y_1)$
- In OpenCV it is implemented with the function `HoughLinesP()`

## What does this program do?

- Loads an image
- Applies a *Standard Hough Line Transform* and a *Probabilistic Line Transform*.
- Display the original image and the detected line in three windows.

## Code

C++ Java Python

The sample code that we will explain can be downloaded from [here](#).

```
"""
@file hough_lines.py
@brief This program demonstrates line finding with the Hough transform
"""
import sys
import math
import cv2 as cv
import numpy as np

def main(argv):

    default_file = 'sudoku.png'
    filename = argv[0] if len(argv) > 0 else default_file

    # Loads an image
    src = cv.imread(cv.samples.findFile(filename), cv.IMREAD_GRAYSCALE)

    # Check if image is loaded fine
    if src is None:
        print('Error opening image!')
        print('Usage: hough_lines.py [image_name -- default ' + default_file + '] \n')
        return -1

    dst = cv.Canny(src, 50, 200, None, 3)

    # Copy edges to the images that will display the results in BGR
    cdst = cv.cvtColor(dst, cv.COLOR_GRAY2BGR)
    cdstP = np.copy(cdst)

    lines = cv.HoughLines(dst, 1, np.pi / 180, 150, None, 0, 0)

    if lines is not None:
```

```

    for i in range(0, len(lines)):
        rho = lines[i][0][0]
        theta = lines[i][0][1]
        a = math.cos(theta)
        b = math.sin(theta)
        x0 = a * rho
        y0 = b * rho
        pt1 = (int(x0 + 1000*(-b)), int(y0 + 1000*(a)))
        pt2 = (int(x0 - 1000*(-b)), int(y0 - 1000*(a)))

        cv.line(cdst, pt1, pt2, (0,0,255), 3, cv.LINE_AA)

linesP = cv.HoughLinesP(dst, 1, np.pi / 180, 50, None, 50, 10)

if linesP is not None:
    for i in range(0, len(linesP)):
        l = linesP[i][0]
        cv.line(cdstP, (l[0], l[1]), (l[2], l[3]), (0,0,255), 3, cv.LINE_AA)

cv.imshow("Source", src)
cv.imshow("Detected Lines (in red) - Standard Hough Line Transform", cdst)
cv.imshow("Detected Lines (in red) - Probabilistic Line Transform", cdstP)

cv.waitKey()
return 0

if __name__ == "__main__":
    main(sys.argv[1:])

```

## Explanation

C++
Java
Python

Load an image:

```

default_file = 'sudoku.png'
filename = argv[0] if len(argv) > 0 else default_file

# Loads an image
src = cv.imread(cv.samples.findFile(filename), cv.IMREAD_GRAYSCALE)

# Check if image is loaded fine
if src is None:
    print ('Error opening image!')
    print ('Usage: hough_lines.py [image_name -- default ' + default_file + '] \n')
    return -1

```

Detect the edges of the image by using a Canny detector:

```

# Edge detection
dst = cv.Canny(src, 50, 200, None, 3)

```

Now we will apply the Hough Line Transform. We will explain how to use both OpenCV functions available for this purpose.

**Standard Hough Line Transform:**

First, you apply the Transform:

```

# Standard Hough Line Transform
lines = cv.HoughLines(dst, 1, np.pi / 180, 150, None, 0, 0)

```

- with the following arguments:
  - *dst*: Output of the edge detector. It should be a grayscale image (although in fact it is a binary one)
  - *lines*: A vector that will store the parameters  $(r, \theta)$  of the detected lines
  - *rho*: The resolution of the parameter  $r$  in pixels. We use 1 pixel.
  - *theta*: The resolution of the parameter  $\theta$  in radians. We use 1 degree (CV\_PI/180)
  - *threshold*: The minimum number of intersections to "detect" a line
  - *sm* and *stn*: Default parameters to zero. Check OpenCV reference for more info.

And then you display the result by drawing the lines.

```

# Draw the lines
if lines is not None:
    for i in range(0, len(lines)):
        rho = lines[i][0][0]
        theta = lines[i][0][1]
        a = math.cos(theta)
        b = math.sin(theta)
        x0 = a * rho
        y0 = b * rho
        pt1 = (int(x0 + 1000*(-b)), int(y0 + 1000*(a)))

```

```
pt2 = (int(x0 - 1000*(-b)), int(y0 - 1000*(a)))

cv.line(cdst, pt1, pt2, (0,0,255), 3, cv.LINE_AA)
```

## Probabilistic Hough Line Transform

First you apply the transform:

```
# Probabilistic Line Transform
linesP = cv.HoughLinesP(dst, 1, np.pi / 180, 50, None, 50, 10)
```

- with the arguments:
  - *dst*: Output of the edge detector. It should be a grayscale image (although in fact it is a binary one)
  - *lines*: A vector that will store the parameters ( $x_{start}, y_{start}, x_{end}, y_{end}$ ) of the detected lines
  - *rho*: The resolution of the parameter  $r$  in pixels. We use **1** pixel.
  - *theta*: The resolution of the parameter  $\theta$  in radians. We use **1 degree** (CV\_PI/180)
  - *threshold*: The minimum number of intersections to **"detect"** a line
  - *minLineLength*: The minimum number of points that can form a line. Lines with less than this number of points are disregarded.
  - *maxLineGap*: The maximum gap between two points to be considered in the same line.

And then you display the result by drawing the lines.

```
# Draw the lines
if linesP is not None:
    for i in range(0, len(linesP)):
        l = linesP[i][0]
        cv.line(cdstP, (l[0], l[1]), (l[2], l[3]), (0,0,255), 3, cv.LINE_AA)
```

Display the original image and the detected lines:

```
# Show results
cv.imshow("Source", src)
cv.imshow("Detected Lines (in red) - Standard Hough Line Transform", cdst)
cv.imshow("Detected Lines (in red) - Probabilistic Line Transform", cdstP)
```

Wait until the user exits the program

```
# Wait and Exit
cv.waitKey()
return 0
```

## Result

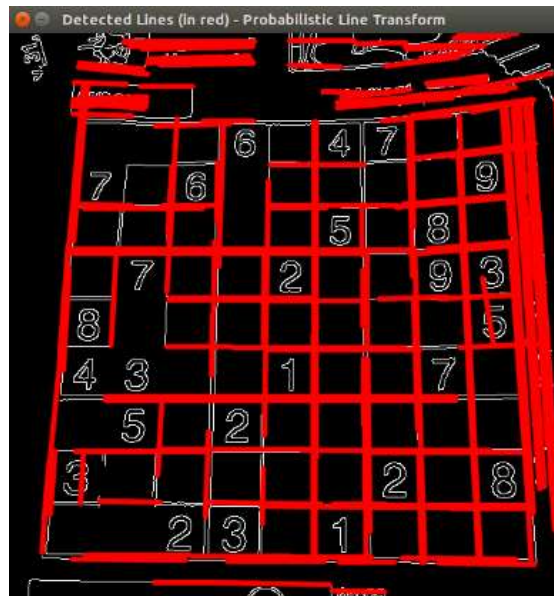
### Note

The results below are obtained using the slightly fancier version we mentioned in the *Code* section. It still implements the same stuff as above, only adding the Trackbar for the Threshold.

Using an input image such as a [sudoku image](#). We get the following result by using the Standard Hough Line Transform:



And by using the Probabilistic Hough Line Transform:



You may observe that the number of lines detected vary while you change the *threshold*. The explanation is sort of evident: If you establish a higher threshold, fewer lines will be detected (since you will need more points to declare a line detected).